Card and error-state spec

One chat request, three cards, and the chat is never blank. This spec turns the hand sketches in `SKETCHES/` into card layouts, field lists, a blank mock response and the error states. The build plan is in `Meeting-Notes.md`.

The happy path

Guest asks
"Plan me a weekend in Kyoto". No login needed.

→

Typing bubble
Three dots while the plan loads.

→

Three cards
Stay, Experience, Food, always in this order.

→

Act on a card
Add to trip, pick a time, browse the menu.

When it goes wrong

Request fails
Error bubble with a red warning sign.

→

Try again
Sends the same request again.

→

Three cards
The chat recovers.

Can't be planned
For example, two countries in 2 days.

→

Go with suggestion
Sends a request the chat can plan.

→

Three cards
For the suggested trip.

Three cards that never look alike

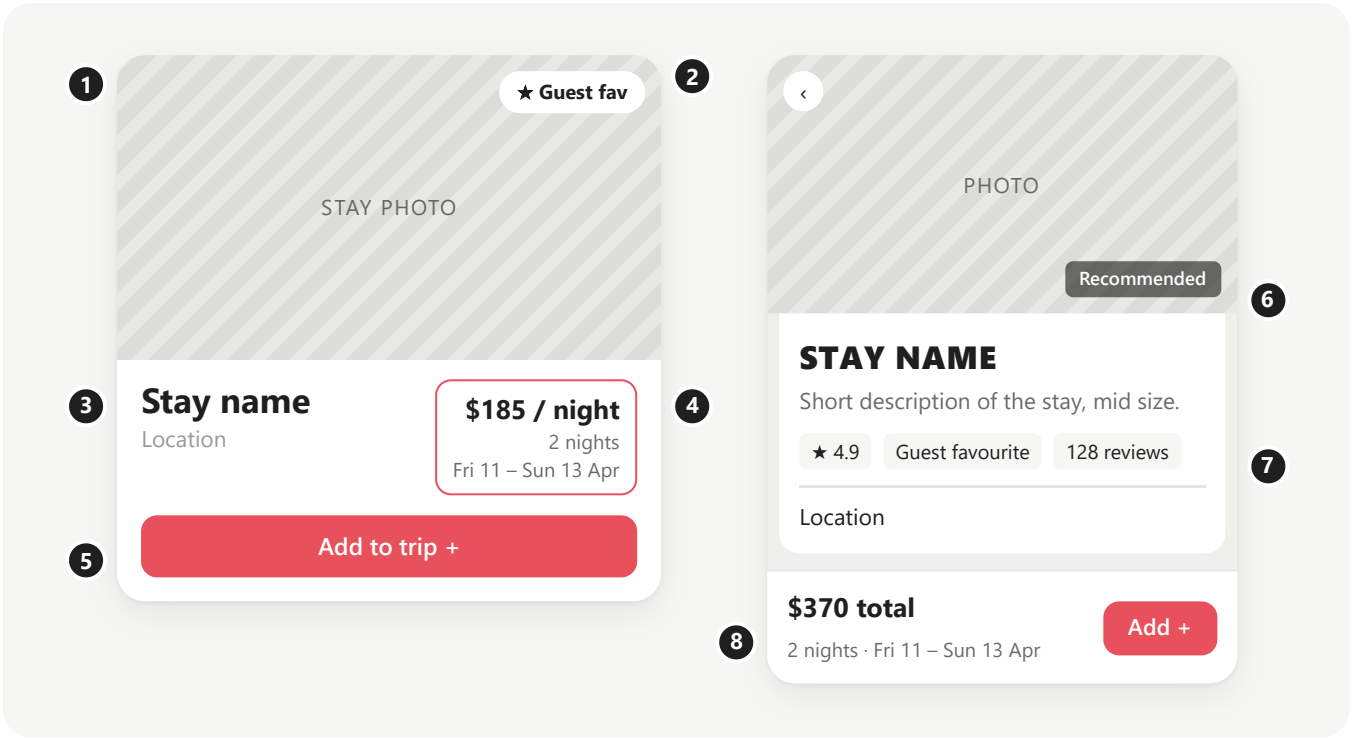
CARD	IMAGE	BIGGEST TEXT	SIGNATURE ELEMENT	ACCENT
Stay	Full-width banner	Stay name	Boxed price + full-width Add button	Coral
Experience	Inset, with duration chip	Experience name	Location card inside the card + split buttons	Violet
Food	Small square, left	Restaurant name	Filter chips + sideways dish row	Amber

Build rules

One component per card	`StayCard`, `ExperienceCard` and `FoodCard` are separate, so the layouts stay truly different.
Stack	TypeScript, Next.js, React and Tailwind. Code on GitHub, one public Vercel link.
Reach	No login. Works on mobile and web.
Never blank	Every error has a button that brings the three cards back.

Stay card

Where the guest sleeps. It leads with a photo and puts the price where the eye lands next. Tapping the card opens the detail screen.

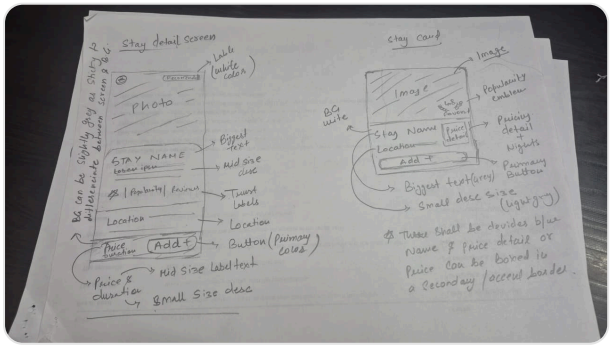


Callouts

1. **Full-width image** across the top of the card
2. **Popularity emblem** on the image
3. **Stay name** is the biggest text. Location sits under it in light grey.
4. **Price, nights + dates** in a box with an accent border. Kyoto weekend: Fri 11 – Sun 13 Apr 2025.
5. **Primary button** on a white card BG
6. **Detail: white label** on the photo, back button top left
7. **Detail: trust labels** rating, popularity, reviews
8. **Detail: sticky bar** on a slightly grey page BG

Fields

FIELD	TYPE	SHOWN AS
`stayName`	text	Biggest text
`location`	text	Small, light grey
`imageUrl`	text	Banner image
`popularityLabel`	text	Emblem
`nightlyPriceUsd`	number	Boxed price
`nights`	number	Under price
`checkIn` / `checkOut`	date	Price box + sticky bar
`description`	text	Detail, mid size
`rating` / `reviewCount`	number	Trust labels



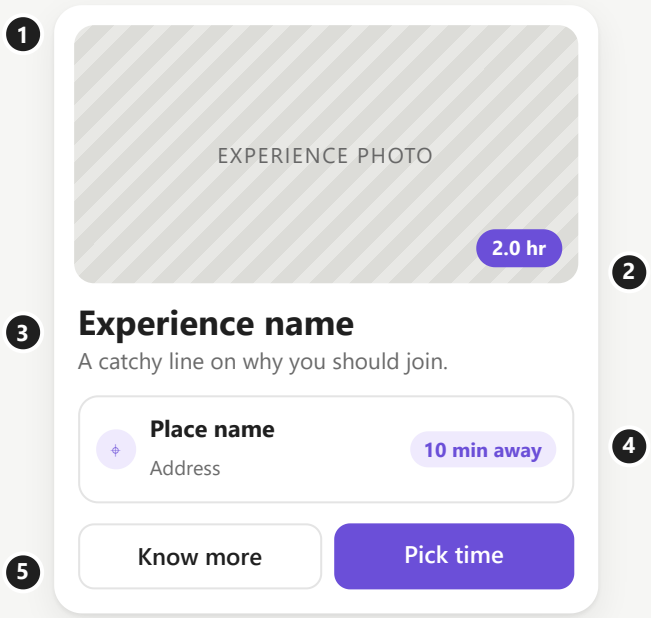
Original sketch: stay card and stay detail screen

The sketch also allows a divider line between the name and the price instead of the box. This spec uses the box because it gives the Stay card a shape the other two don't have.

CARD 2 OF 3

Experience card

Something to do. It tells the guest how long it takes and how far it is from the stay, then lets them read more or book a slot.

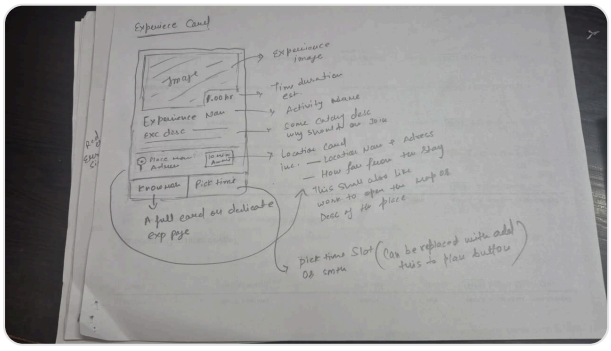


Callouts

1. **Inset image** with rounded corners, set in from the card edge
2. **Duration chip** estimated time, sits on the image
3. **Experience name** is the biggest text, with one catchy line under it
4. **Location card** place, address and distance from the stay. Tap it to open the map or place info.
5. Split footer **Know more** opens the full experience page. **Pick time** opens a slot picker (could later be "Add to plan").

Fields

FIELD	TYPE	SHOWN AS
`experienceName`	text	Biggest text
`description`	text	Catchy line
`imageUrl`	text	Inset image
`durationHours`	number	Chip on image
`placeName`	text	Location card, line 1
`address`	text	Location card, line 2
`minutesFromStay`	number	"10 min away" chip
`timeSlots`	list of text	Pick time sheet



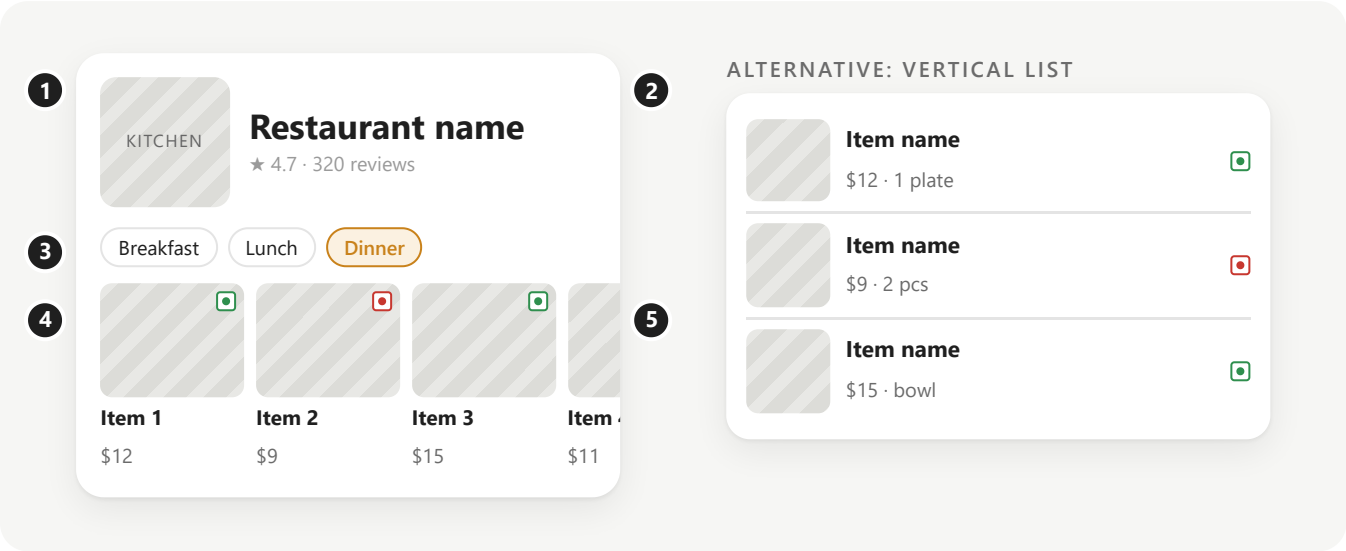
Original sketch: experience card

The location card is the main thing that sets this card apart. It's the only card with a second card inside it, and it links the experience back to the stay.

CARD 3 OF 3

Food card

Where to eat. The dishes are the hero here, not the building: a small kitchen photo, quick filters, then a row of dishes to browse.

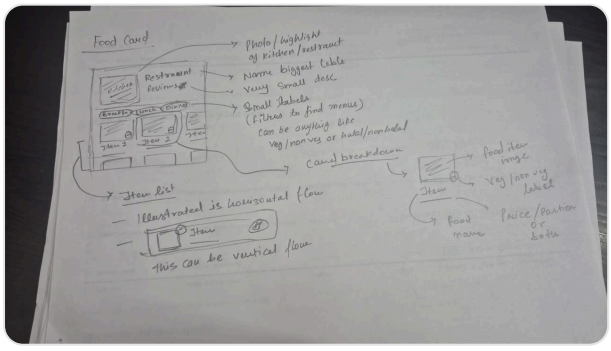


Callouts

1. **Small square photo** of the kitchen or a table, left of the name
2. **Restaurant name** is the biggest text. Reviews sit under it, very small.
3. **Filter chips** meal times, or veg / non-veg, halal / non-halal
4. **Dish row** scrolls sideways (a vertical list also works)
5. **Diet dot** green = veg, red = non-veg. Each dish shows its name and price and/or portion.

Fields

FIELD	TYPE	SHOWN AS
`restaurantName`	text	Biggest text
`rating` / `reviewCount`	number	Very small
`imageUrl`	text	Square photo
`filters`	list of text	Chips
`menuItems[].itemName`	text	Dish name
`menuItems[].imageUrl`	text	Dish thumbnail
`menuItems[].dietLabel`	text	Veg / non-veg dot
`menuItems[].priceUsd`	number	Under the name
`menuItems[].portion`	text	Optional



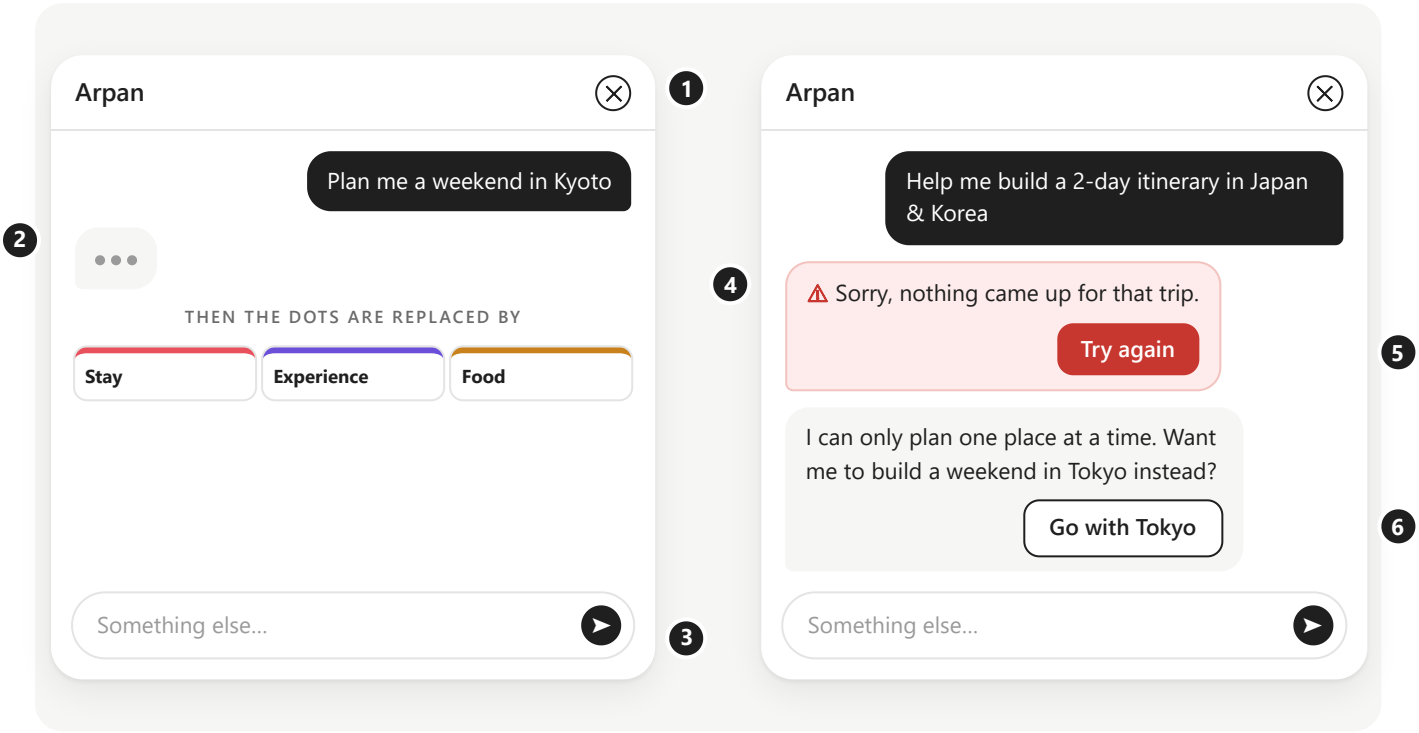
Original sketch: food card and item breakdown

No banner image: this is the only card whose picture is a small square, which keeps it visibly different from the Stay card even at a glance.

CHAT WINDOW

Loading and error states

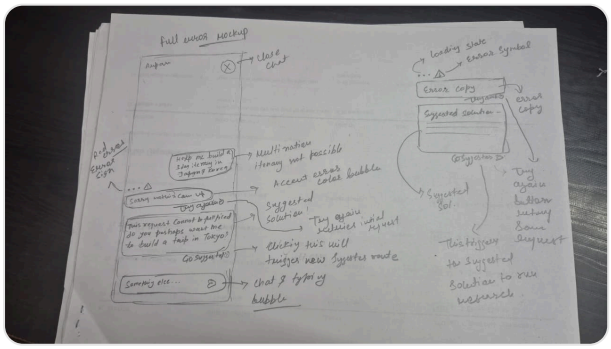
The thread is never blank. When the plan can't be shown, the guest gets a clear reason and a button that leads back to the three cards.



1. **Header** guest name, close-chat button
2. **Typing bubble** the loading state, replaced by the three cards
3. **Input** "Something else..." and a send button
4. **Error bubble** error accent colour and a red warning sign
5. **Try again** resends the original request
6. **Suggested solution** "Go with..." sends the suggested request instead

PART	VALUE
Error copy	Short, plain, no blame. Says what happened.
Try again	Inside the error bubble, bottom right. Sends the same request again.
Suggestion	Shown only when the request can't be planned as asked (for example, two countries in 2 days).
Forced failure	Add `?fail=once` to the address. The first request fails and every request after it works.

> **Rule:** both buttons end with the Stay, Experience and Food cards showing, in that order.



Original sketch: full error mockup

Steps to fire the forced failure

1. Open the live link with `?fail=once` on the end.
2. Type "Plan me a weekend in Kyoto" and press Send.
3. The typing bubble shows, then the error bubble appears.
4. Tap **Try again**. The three cards appear in order.

DATA

Mock response

Fake data that stands in for the assistant's real answer while the prototype is built. `cardType` tells the chat which component to draw. The values are blank on purpose, apart from the stay dates and nights, which are fixed for the Kyoto weekend (Fri 11 – Sun 13 Apr 2025).

ITEM 1
stay
11 fields → `StayCard`

ITEM 2
experience
8 fields → `ExperienceCard`

ITEM 3
food
6 fields + 5 per dish → `FoodCard`

```
[
  {
    "cardType": "stay",
    "details": {
      "stayName": "",
      "location": "",
      "imageUrl": "",
      "popularityLabel": "",
      "nightlyPriceUsd": null,
      "nights": 2,
      "checkIn": "2025-04-11",
      "checkOut": "2025-04-13",
      "description": "",
      "rating": null,
      "reviewCount": null
    }
  },
  {
    "cardType": "experience",
    "details": {
      "experienceName": "",
      "description": "",
      "imageUrl": "",
      "durationHours": null,
      "placeName": "",
      "address": "",
      "minutesFromStay": null,
      "timeSlots": []
    }
  },
  {
    "cardType": "food",
    "details": {
      "restaurantName": "",
      "rating": null,
      "reviewCount": null,
      "imageUrl": "",
      "filters": [],
      "menuItems": [
        {
          "itemName": "",
          "imageUrl": "",
          "dietLabel": "",
          "priceUsd": null,
          "portion": ""
        }
      ]
    }
  }
]
```

Check before hand-in

- ✓ Exactly 3 items, in the order stay, experience, food
- ✓ Each item has a `cardType` and a `details` object
- ✓ Every field name matches the card's field table
- ✓ Every value is blank (`""`, `null` or `[]`) except the stay dates and nights
- ✓ The three cards differ in image, biggest text and signature element